

Network Architecture Diagram

Product: KRC Pune — VR / Amenities Sales Experience Application **Vendor:** Fute Services **Document date:** 16 July 2026 **Assessed at commit:** 5876920 , branch main

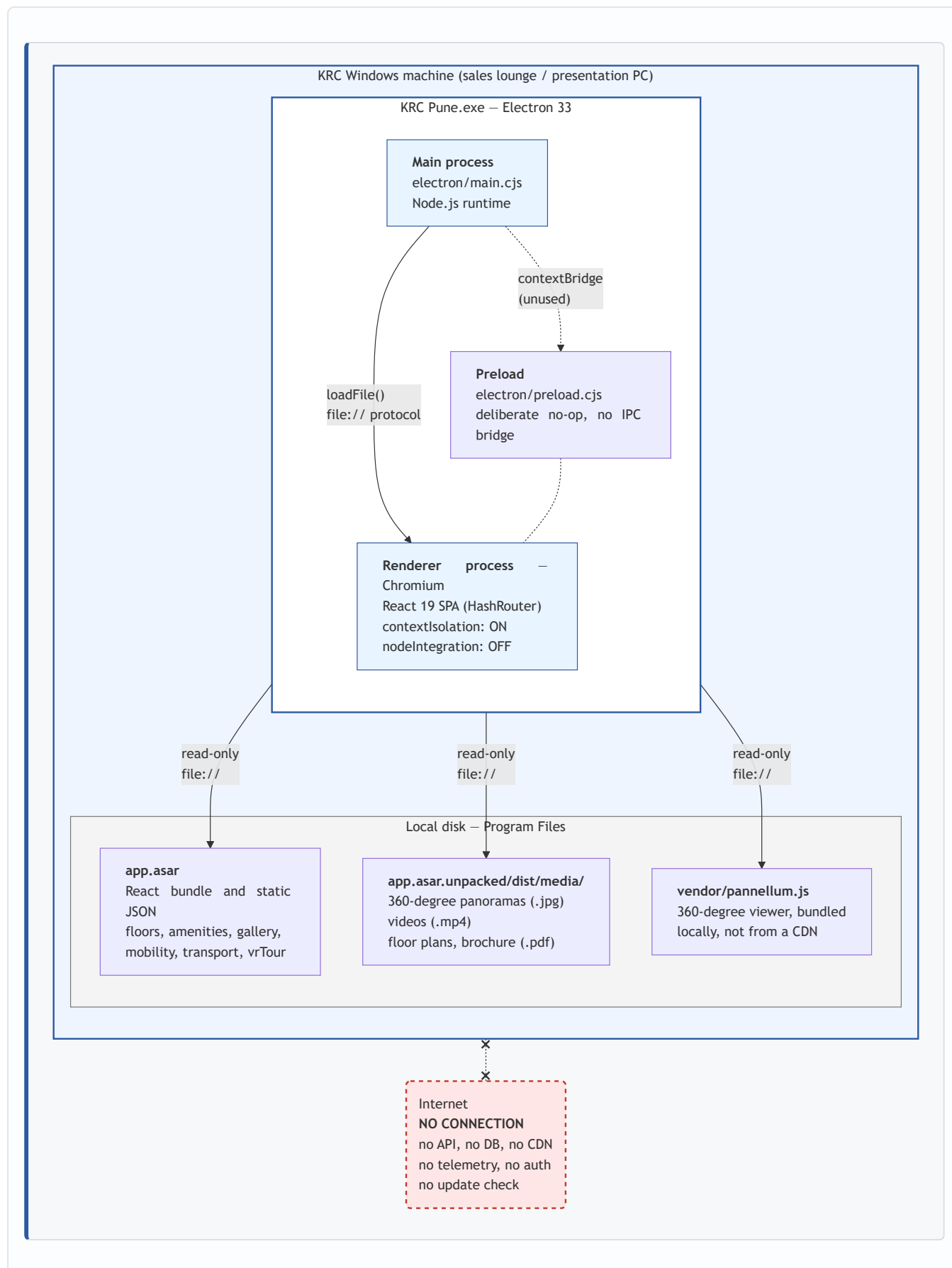
1. Summary

KRC Pune is a standalone Windows desktop application with no runtime network dependency. Once installed it does not connect to any server, API, database or third-party service. It runs correctly on a permanently air-gapped machine.

There is therefore no conventional application topology to draw. There are no tiers, no load balancers, no DMZ and no data centre, because there is nothing for the application to talk to. The only network activity in the product's lifecycle happens before delivery, while the installer is being built and distributed. That part is diagrammed separately in section 3.

2. Runtime architecture on the KRC machine

The diagram below is the complete picture of the application as it runs at KRC.



2.1 Runtime ports

Direction	Port	Protocol	Purpose
Inbound	None	—	The application opens no listening socket
Outbound	None	—	The application initiates no connection

No firewall rule, proxy exception or whitelisting is needed for the application to work.

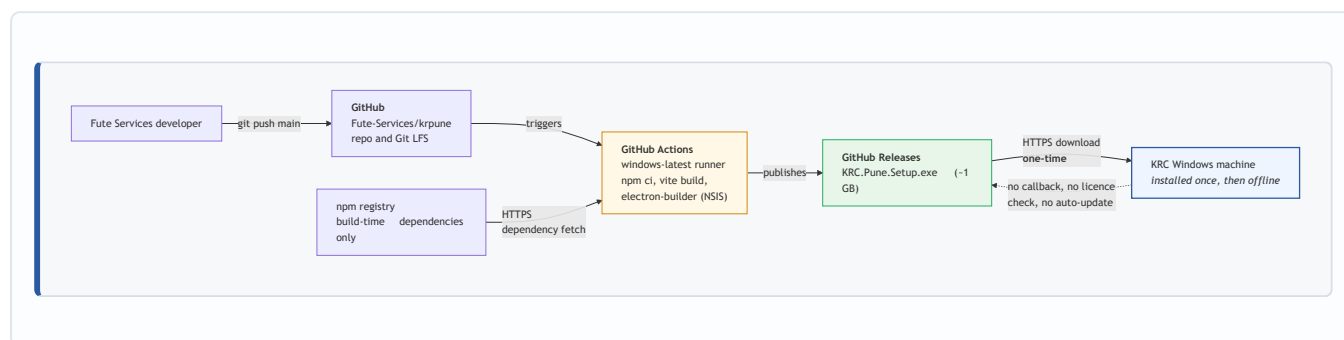
2.2 The one deliberate exception

The Electron main process contains two guards that hand any external URL to the user's default browser and refuse to load it inside the application window. The first is a window-open handler that calls `shell.openExternal(url)` and then denies the in-app load. The second intercepts navigation: any URL that is not `file://` is cancelled and passed to the system browser instead.

Any traffic that results belongs to the system browser, not to KRC Pune, and only happens when a user clicks something. In the current build the only such link is a Microsoft Teams button, and it is commented out and not rendered, so no external link is reachable in the shipped interface at all.

3. Build and distribution

Network activity exists only in the pipeline that produces the installer. This runs on Fute Services and GitHub infrastructure and stops the moment the installer is handed over.



3.1 Build-time third parties

Service	Role	Data exposed	Reaches KRC runtime
GitHub (repository, Actions, Releases)	Source control, CI, installer hosting	Source code and build artifacts	No
npm registry	Build-time dependency resolution	None; public packages	No. Dependencies are compiled into the bundle

No KRC data passes through this pipeline at any point. It carries product content only: renders, plans and marketing media.

4. Points we want to flag

These are disclosed for completeness. Each is a distribution concern rather than a runtime network concern, but your team should know about them.

The installer is not code-signed. Windows SmartScreen shows an unknown-publisher warning and users are currently told to click through it via More info, then Run anyway. That trains people to bypass a security control, and it means the integrity and authenticity of the installer cannot be verified cryptographically. The fix is an OV or EV code-signing certificate. Usefully, the CI pipeline already supports this: it reads the `WINDOWS_CERT_BASE64` and `WINDOWS_CERT_PASSWORD` repository secrets and signs automatically once they are set, so no code change is needed.

The installer is distributed through a public GitHub Release with a stable link that always points at the newest build. Anyone holding that link can download the build and the marketing media inside it. If KRC would prefer otherwise, we can move to a private or authenticated channel, or hand-deliver the installer.

There is no auto-update mechanism, so a security fix requires manual redistribution and reinstallation. This is a reasonable trade-off for an offline kiosk application, but it does mean we should agree a patch-delivery process with your team rather than leave it undefined.

5. Attestation

The runtime architecture described here was verified by scanning the compiled production bundle (`code/dist/assets/`) for outbound hosts. No external service endpoint is present. This is a Fute Services self-declaration based on source and build inspection at commit `5876920` . It is not an independent third-party audit.

Prepared by *[Name, Title]*, Fute Services, 16 July 2026